

Vetor(ARRAY)

Recurso que permite a criação de uma variável multivalorada, sendo ela uma estrutura simples dentre as estruturas de dados. Muitas vezes ela é formada por dados do mesmo tipo, e os dados são acessados através de sua posição, ou seja, um índice que sempre será do tipo inteiro.

Durante a sua criação, será definido a quantidade de elementos, de acordo com a figura abaixo:

Vet	0	1	2	3	4	5	6	7
	55	76	26	64	26	80	71	46

Exemplo:

```
<script>
    const MAX = 10;
    //decloarando o vetor
    var v = [];
    var i,valor,encontrou;

    // Grava os elementos no vetor
    for ( i = 0; i < MAX; i++) {
        valor = Number(window.prompt(`Digite o ${i}º elemento do vetor:`));
        v.push(valor);
    }

    // Lista os elementos do vetor
    document.write("Valores no vetor:"+"<br>");
    for (i = 0; i < MAX; i++) {
        document.write(`${i}: ${v[i]}`+'<br>');
    }

    // Mostra onde foi encontrado o valor 7
    document.write("\nPosições onde o valor 7 foi encontrado:"+"<br>");
    encontrou = false;
    for ( i = 0; i < MAX; i++) {
        if (v[i] == 7) {
            document.write(`O valor 7 foi encontrado na posição ${i}`+"<br>");
            encontrou = true;
        }
    }

    if (!encontrou) {
        document.write('<br>')
```

```
        document.write("0 valor 7 não foi encontrado no vetor.");  
    }  
  
</script>
```

Exemplo array de números

```
<script>  
    var array=[1,2,3,4]  
    var i  
    for (i=0;i<4;i++){  
        document.write(array[i]+"<br>")  
    }  
  
</script>
```

Objetos em Javascript

Em **JavaScript**, objetos são estruturas que permitem armazenar dados de forma organizada usando **pares de chave e valor**.

Em vez de guardar informações separadas, você agrupa tudo em um único “pacote”.

Exemplo de objeto produto

```
script>

// Criando um objeto para um único produto
var produto = {
  nome: "Notebook",
  preco: 3500,
  emEstoque: true
};

// Exibindo no documento
document.write("Produto: " + produto.nome + "<br>");
document.write("Preço: R$ " + produto.preco + "<br>");
document.write("Em Estoque: " + produto.emEstoque + "<br>");

</script>
```

Vetor(array) de objetos exemplo

Podemos também armazenar objetos dentro de um array. Observe o exemplo:

```
<script>
// Criando uma lista (array) de objetos
var i
var listaDeProdutos = [
  {
    nome: "Notebook",
    preco: 3500,
    emEstoque: true
  },
  {
    nome: "Mouse",
    preco: 80,
    emEstoque: true
  },
  {
```

```

        nome: "Teclado",
        preco: 150,
        emEstoque: false
    }
];

// Exibindo a lista no console
document.write("Lista de Produtos:"+"<br>");
for (i=0;i<3;i++){
    document.write("Produto: " + listaDeProdutos[i].nome + "<br>");
    document.write("Preço: R$ " + listaDeProdutos[i].preco + "<br>");
    document.write("Em Estoque: " + listaDeProdutos[i].emEstoque + "<br><br>");
}

</script>

```

Podemos também deixar a cargo do usuário digitar a lista. Veja o exemplo:

```

<script>
    var listaDeProdutos = [];
    var quantidade = 3; // quantidade de produtos a serem inseridos

    for (var i = 0; i < quantidade; i++) {
        var vnome = window.prompt("Digite o nome do produto " + (i + 1) + ":");
        var vpreco = Number(window.prompt("Digite o preço do produto " + (i + 1) +
        ":"));
        var vemEstoque = window.prompt("O produto está em estoque? (sim/não):");

        listaDeProdutos.push({
            nome: vnome,
            preco: vpreco,
            emEstoque: vemEstoque
        });
    }

    document.write("<h3>Lista de Produtos:</h3>");
    for (var i = 0; i < quantidade; i++) {
        document.write("Produto: " + listaDeProdutos[i].nome + "<br>");
        document.write("Preço: R$ " + listaDeProdutos[i].preco + "<br>");
        document.write("Em Estoque: " + listaDeProdutos[i].emEstoque+"<br><br>");
    }
</script>

```

Algoritmo de ordenação bubble sort

O Bubble sort é um algoritmo de ordenação simples que organiza listas ou arrays comparando elementos adjacentes e trocando-os de lugar se estiverem na ordem errada. Esse processo repete-se várias vezes, “flutuando” os maiores valores para o final da lista (como bolhas).

Exemplo:

```
<script>
    const MAX = 10;
    var v = [];
    var i,j,aux;
    // Grava os elementos no vetor
    for ( i = 0; i < MAX; i++) {
        valor = Number(window.prompt(`Digite o ${i}º elemento do vetor:`));
        v.push(valor);
    }

    // Lista os elementos do vetor original
    document.write("Valores no vetor:"+"<br>");
    for (i = 0; i < MAX; i++) {
        document.write(`${i}: ${v[i]}`+'<br>');
    }

    // ordena os elementos do vetor
    for(i=0;i<MAX;i++){
        for(j=0;j<MAX-1;j++){
            if (v[j]>v[j+1]){
                aux = v[j+1]
                v[j+1]= v[j]
                v[j]=aux
            }
        }
    }

    // Lista os elementos do vetor ordenado
    document.write("Valores no vetor ordenado:"+"<br>");
    for (i = 0; i < MAX; i++) {
        document.write(`${i}: ${v[i]}`+'<br>');
    }
</script>
```

Funções em JavaScript

Funções nada mais são do que blocos de códigos que executam uma determinada tarefa. Porém as funções somente são executadas mediante uma chamada específica a elas. Utilizamos funções quando percebemos a necessidade de se repetir um código mais do que uma vez, ou seja, quando precisamos reaproveitar código. Ao utilizar funções é importante lembrar que o ideal é criá-las para fins específicos, ou seja, para executar uma determinada tarefa. Elas podem ser de dois tipos:

Funções que retornam

São aquelas que apresentam o “return”, retornando algo para a aplicação. Exemplo:

```
<script>

    function calcarea(p1,p2){
        var res;
        res= p1*p2;
        return res;
    }

    // exemplo de função pois o código apresenta o return
    var base, altura,resultado;
    base = Number(window.prompt('digite o primeiro valor'));
    altura = Number(window.prompt('digite o segundo valor'));
    resultado=calcarea(base,altura);
    document.write(resultado);

</script>
```

Funções que não retornam(procedimentos)

São aquelas que não apresentam o “**return**”, não retornando nada para a aplicação. Exemplo:

```
<script>

    function calcarea(p1,p2){
        var res;
        res= p1*p2;
        return res;
    }

    // exemplo de função pois o código apresenta o return
    var base, altura,resultado;
    base = Number(window.prompt('digite o primeiro valor'));
    altura = Number(window.prompt('digite o segundo valor'));
    resultado=calcarea(base,altura);
    document.write(resultado);

</script>
```

Funções anônimas

Funções anônimas em JavaScript são funções definidas sem um nome identificador, geralmente criadas no local de uso e atribuídas a variáveis ou passados como argumentos(call-backs). Elas são ideais para tarefas únicas, manipulação de eventos e para evitar conflitos de escopo, sendo amplamente utilizadas em códigos modernos.

Exemplo 01:

```
<script>
    let mostrarMensagem = function () {
        alert("Olá!");
    };

    mostrarMensagem(); // Isso chama a função e mostra o alerta
</script>
```

Exemplo 02:

```
<script>
    let funcao = function () {

        soma(1, 2);
    }
    function soma(p1,p2){
        let res= p1+p2
        document.write("a soma é"+res)
    }

    funcao()

</script>
```

Recursividade

Recursividade é um conceito fundamental na computação e matemática onde uma função chama a si mesma para resolver um problema. Ela divide um problema complexo em subproblemas menores e mais simples, repetindo o processo até atingir uma condição de parada que encerra a recursão e evita loops infinitos.

Exemplo 01:

```
<script>
    function exibir(n) {
        document.write(n+"<br>")
        // Condição de parada da função
        if ( n == 1) {
            return 1;
        }
        // Passo recursivo:
        return exibir(n-1)
    }

    // Exemplo de uso
    exibir(5);
</script>
```


Exemplo 02:

```
<script>
  function fatorial(n) {
    // Caso base: termina em 1
    if ( n == 1) {
      return 1;
    }
    // Passo recursivo
    return n * fatorial(n - 1);
  }

  // Exemplo de uso
  document.write(fatorial(5)); // Saída: 120
</script>
```